

Syllabus

Detailed Curriculum

Module No.	Unit No.	Contents	No. of Hrs.	CO
1	Introduction to analysis of algorithm		06	CO 1
	1.1	Performance analysis, space and time complexity, Growth of function-Big-Oh; Omega; Theta Notation, Analysis of insertion sort, Introduction to randomized algorithm, Solving Recurrence Problems by Substitution Method, Recursion Tree Method, Masters Method.		
2	Algorithm Design Techniques			
	2.1	Divide and Conquer Technique General method, Finding minimum and maximum algorithm and analysis, Analysis of Merge sort, Quick sort and Heap Sort, Strassen's Matrix Multiplication.	06	CO 2
	2.2	Greedy Technique General method, Knapsack problem, Job Scheduling with deadlines, Minimum cost spanning trees-Kruskal's and Prims algorithm, Single source shortest path.	06	CO 2
	2.3	Dynamic Programming Technique General method, Multistage graphs, 0/1 knapsack, Travelling salesman problem, Single source shortest path, All pairs shortest path, Matrix chain multiplication.	07	CO 2
	2.4	Backtracking Technique General method, Sum of subsets, 8-queens problem, Graph coloring.	04	CO 2
	2.5	Branch and Bound General method, 0/1 Knapsack, 15 Puzzle Problem.	04	CO 2
3	String Matching Algorithms		06	CO 3
	3.1	The naïve string-matching Algorithms, The Knuth-Morris-Pratt algorithm, Longest common subsequence.		
4	Non-deterministic Polynomial Algorithms		06	CO 4
	4.1	Polynomial time, Polynomial time verification, NP Completeness and reducibility, NP Completeness proof: Vertex Cover Problem.		
#Self-Learning Topic- Rod cutting algorithm, randomization algorithms				
Total			45	--

Module 1

Introduction to AOA

Definition: An Algorithm is a finite sequence of instructions. Each instruction has a clear meaning and can be performed in finite time.

Criteria for an algorithm:

1. Input: zero or more information is given
2. Output: At least one result is produced
3. Each instruction is unambiguous
4. Finiteness: For all cases the algorithm should terminate after a finite number of steps
5. Feasibility: Each instruction should be basic enough to be carried out on paper.

The performance of a program is measured in terms of its time and space complexity.

Time complexity

The time needed by an algorithm to run expressed as a function of its size is called the time complexity of the algorithm. Simply put, the time complexity of a program is the amount of computer time it needs to run to completion.

Has three cases:

- Best
- Worst
- Average

Space complexity

The space complexity of a program is the amount of memory it needs to run.

- Instruction space: Instruction space is the space needed to store the compiled version of the program instructions.
- Data space: Data space is the space needed to store all constant and variable values.
- Environment stack space: used to save information needed to resume execution of partially completed functions.

Complexity of an algorithm

Definition: The function that gives the running time/storage space required for the algorithm based on its input data size (n)

Two ways to calculate

1. Step count method
2. Asymptotic Notations

Step count method

Rules:

1. For all declarations, count=0
2. Comments, count=0
3. For all assignment operations, count=1
4. For all return operations, count=1
5. When function is polynomial, ignore all lower powers and only consider the highest polynomial
Example: $f(n) = 6n^3 + 10n^2 + 15n + 3 \Rightarrow 6n^3$
6. Ignore constants. In the above example, $f(n) = n^3$

Examples:

1.

```
Algorithm sum (int a[], int n
```

```
    s = 0;
```

```
    for(i=0; i<n; i++)
```

```
    {
```

```
        s=s + a[i];
```

```
    }
```

```
    return s;
```

Time complexity:

Assignment: count=1

Run N+1 times (last loop updates and then stops running)

Assignment: count+=1 for each run= n times

Return: count= 1

Total: $1+n+1+n+1 = 2n+3 = O(n)$

Space complexity: a[] = n words

n=1 word, s=1 word, i= 1 word

Max: $1+1+1+n = 3+n = O(n)$

```
Algorithm addMat (int a[][], int b[][])
```

```
{ int c[][];
```

```
  for(i=0; i<n; i++) {
```

```
    for(j=0; j<n; j++) {
```

```
      c[i][j] = a[i][j] + b[i][j]
```

```
    }
```

```
  }
```

```
}
```

```

Algorithm matMul (int a[][], int b[][])
{ int c[][];
  for(i=0; i<n; i++) {
    for(j=0; j<n; j++) {
      c[i][j] = 0;
      for(k=0; k<n; k++){
        c[i][j] = a[i][k] * b[k][j]
      }
    }
  }
}

```

Example: loops (By tracing)

5.

```

for(i=0; i<n; i++) {
  for(j=0; j<i; j++) {
    statements;
  }
}

```

$$1 + 2 + 3 + 4 + \dots + n = \frac{n(n+1)}{2}$$

$$T(n) = 1 + 2 + 3 + 4 + \dots + n - 1 = \frac{(n-1)(n)}{2} = O(n^2)$$

Time Complexity		
i	j	statements
0	0	0
1	0	1
	1	
2	0	2
	1	
	2	
3	0	3
	1	
	2	
	3	
...	...	...
N	0 to n-1	n

Example 5: loops (By tracing)

```
6.  p=0;
    for(i=1; p<=n; i++) {
        p=p+i;
    }
}
```

$$\begin{aligned}
 &= 1+2+3+4+\dots+k > n \\
 &= \frac{k(k+1)}{2} > n \\
 &= \frac{k^2 + k}{2} > n \\
 &\cong \frac{k^2}{2} > n \\
 &\quad \mathbf{k = \sqrt{n} = O(n)}
 \end{aligned}$$

Time Complexity		
i	p	statements
1	0+1	1
2	1+2	1
3	1+2+3	1
4	1+2+3+4	1
5	1+2+3+4+5	1
6	1+2+3+4+5+6	1
k	1+2+3+4+...+k	???

Correction: time complexity here is $O(\sqrt{n})$ since $k^2 > n \Rightarrow k > \sqrt{n}$

And the number of statements is K, therefore, time complexity = $\sqrt{n}$

```
for(i=1; i<n; i=i*2) {
    statements;
}
}
```

```
for(i=n; i>=1; i=i/2) {
    statements;
}
}
```

answer: $\log n$

```
for(i=0; i*i<n; i++) {
    statements;
}
}
```

Answer: $O(\sqrt{n})$

```

for(i=1; i<n; i=i*2) {
    p++;
}
for(j=1; j<p; j=j*2){
    statements;
}

```

Answer: $O(\log(\log n))$

Correction: the loops are nested

Asymptotic Notations

Three types

1. Big O (worst)
2. Big omega (best)
3. Big theta (average)

In numerical terms,

- The Big O notation gives the tight upper bound of the given function.
At larger values of n , upper bound of $f(n)$ is $g(n)$.
 $f(n) = O(g(n))$
- Big omega notation gives the tight lower bound of the given function.
at larger values of n , lower bound of $f(n)$ is $g(n)$.
 $f(n) = \Omega(g(n))$
- Average running time of an algorithm is always between lower bound and upper bound. It is represented by big Theta.

Analysis of insertion sort, Introduction to randomized algorithm, Solving Recurrence Problems by Substitution Method, Recursion Tree Method, Masters Method.

Pending

Masters method

$$T(n) = a \log_b \frac{n}{b} + f(n)$$

case 1:

$$f(n) < n^{\log_b a}$$

$$T(n) = O(n^{\log_b a})$$

case 2:

$$f(n) = n^{\log_b a}$$

$$T(n) = O(\log n \times n^{\log_b a})$$

case 3:

$$f(n) > n^{\log_b a}$$

$$T(n) = O(f(n))$$

Module 2

Divide and Conquer

All codes:

<https://github.com/ashwera/SYNotes/tree/master/Semester%202/Analaysis%20of%20Algorithms/aoa%20ost>

Meaning: Split original problem into smaller similar subproblems then solve. Solution of these small independent subproblems are combined to obtain the solution of the whole problem.

Three steps: Divide, solve, combine.

Complexity is given by recurrence relations.

The general recurrence relation for divide and conquer is given as follows:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Here, $T(n/b)$ is the time required to solve each subproblem

$f(n)$ is the time required to combine the solutions of all subproblems

a = number of subproblems, b = factor by which input size reduces

Binary search

There are two approaches:

- Iterative
- Recursive

Rules

1. Used only on sorted containers
2. We compare "key" with an item in the middle position. If they are equal, search ends successfully.
3. If the middle is more than the key, we reduce the search space by removing the upper half
4. If the middle is less than the key, we remove the lower half of the search space.

Iterative algorithm

```
Algorithm IBinaryS(arr[ ], start, end, key){
```

```
    int mid;
    while(start<=end){
        mid = (start + end)/2;
        if (arr[mid] == key)
            return 1;
        if (arr[mid]<key)
```

```

        start = mid+1;
    else
        end = mid-1;
    }
    return 0;
}

```

Recursive

```

Algorithm RBinaryS(arr[ ], start, end, key){
    int mid;
    if (start > end) { return 0; }
    else
        mid = (start + end)/2;
        if (key == arr[mid])
            return (mid);
        else
            if (key < arr[mid]){
                RBinaryS(arr[],key, start, mid-1) //apply BS on lower half
            }
            else
                RBinaryS(arr[],key, mid+1, end) //apply BS on upper half
    }
}

```

Minimum/Maximum element search

Iterative

What is happening:

1. Assign first element as both min and max
2. Traverse through the array once
3. When we find a smaller than min element, we reassign that to min
4. When we find a larger than max element, we reassign that to max
5. Time complexity: $O(n)$ as we go thru all elements of the array once.

```

Algorithm MinMax(a[ ], n, max, min){
    max=min=a[1];
    for(i=2 to n){
        if(a[i]> max) max=a[i];
        if (a[i]< max) min=a[i];
    }
}

```



```
}
```

Simple algorithm: needs $2(n-1)$ comparisons. $N-1$ for minimum and $n-1$ for maximum.

Recursive

```
Algorithm MinMax(a[ ], l, h, max, min) {
    if(l==h){
        max=min=a[l];
    }
    else if (h-l==1){ //h and l are adj indices, so only 2 numbers and two
        comparisons
        if(a[l]>=a[h]){
            max=a[l];
            min=a[h];
        }
        else{
            max=a[h];
            min=a[l];
        }
    }
    else{
        Mid = (l+h)/2;
        MinMax(a, l, mid, max, min); //find minmax of left half
        MinMax(a, mid+1, h, max, min); //find minmax of right half
        if(max< max1) then max=max1; //make the larger max the global max
        if (min>min1)then, min = min1; //same for global min
    }
}
```

Space complexity of recursively finding min max is worse than linear: we need $\log n+1$ space.

Time complexity in worse case is also $O(n)$

Therefore, for finding min and max elements, the recursive approach is worse.

Time complexity analysis

Recursive MinMax

Base case:

$n = 1$ $T(1) = 0$

$n = 2$ one comparison $T(2) = 1$

Recurrence:

- split in two

$$T(n) = T(n/2) + T(n/2) + 2$$

↑
2 comparisons

$$= 2T(n/2) + 2$$
$$T(n/2) = 2T(n/4) + 2$$
$$T(n) = 2(2T(n/4) + 2) + 2$$
$$= 4T(n/4) + 6$$

after k steps:

$$T(n) = 2^k T(n/2^k) + 2(2^k - 1)$$

stop when $n/2^k = 1 \Rightarrow 2^k = n$

$$k = \log_2 n$$
$$T(n) = n T(1) + 2(n-1)$$
$$T(1) = 0 \quad \text{"base case"}$$
$$T(n) = n + 2n - 2$$
$$= 3n - 2$$
$$\therefore T(n) = O(n)$$

Merge Sort

What is done:

1. List of N elements is divided into 2 lists of $n/2$ elements. Takes constant time: $O(1)$ as it only computes the middle of the array.
2. Each half is sorted independently. $T(n/2) + T(n/2)$
3. Two lists are combined in one: takes $O(n)$ time.

Algo:

```
mergeSort(arr[], low, high)
```

```
// arr is array, low is left sub-list, high is right sub-list
```

```
{
```

```

    if (low < high) {
        mid = (low + high) / 2; //finding the middle point to divide. O(1)
        mergeSort(arr, low, mid); //sorting left half o(n/2)
        mergeSort(arr, mid + 1, high); //sorting right half o(n/2)
        merge(arr, low, mid, high); //merging both halves o(n)
    }
}
//this is a recursive function to keep sorting the array.
Can be understood by making a recursion tree.

```

```

void merge(int arr[], int low, int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = low;

    /* create temp array */
    int temp[5];

    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j]) {
            temp[k] = arr[i];
            i++;
            k++;
        } else {
            temp[k] = arr[j];
            j++;
            k++;
        }
    }

    while (i <= mid) {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= high) {

```

```
        temp[k] = arr[j];
        j++;
        k++;
    }
//what this does:
//we have two sorted arrays we want to merge into one
//so we maintain two pointers for two arrays and compare their values one by one
// as per desired order, we increment pointer in temp array and copy values from
sorted array
//after comparison, we copy all remaining elements from either left or right
array if any is left behind.

    for (int k = low; k <= high; k++) {
        arr[k] = temp[k];
    }
}
```

Time complexity analysis

Merge sort

Recurrence :

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$\uparrow$ $\uparrow$
divide in 2 merge step

Base case $T(1) = O(1)$

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$
$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + \frac{n}{2}$$
$$\therefore T(n) = 2\left(2T\left(\frac{n}{4}\right) + \frac{n}{2}\right) + n$$
$$= 4T\left(\frac{n}{4}\right) + n + n$$
$$= 4T\left(\frac{n}{4}\right) + 2n$$

After k steps

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kn$$

we stop when $\frac{n}{2^k} = 1$

$$n = 2^k$$
$$k = \log_2 n$$
$$T(n) = n T(1) + \log_2 n \cdot n$$

but $T(1) = 1$

$$T(n) = n + n \log n$$
$$\therefore T(n) = O(n \log n)$$

Quick sort

What happens:

1. Array is divided into subarrays
Splitting is based on pivot
2. Left subarray: $\leq$ pivot
Right: $>$ pivot
3. Recursively sort
4. Combine all sorted elements

We randomly choose the first element as pivot.

Set $i=1$ and $j=\text{last}$

Then, we move forward the i pointer till $v[i] \leq$ pivot fails

Move j backward till $v[j] >$ pivot condition fails

Here, when $\text{pivot}=50$, i and j are at 90 and 40 respectively.

Low				High			
50	30	10	90	80	20	40	70
Pivot			i	j			

Since $v[i]$ is more than pivot and $v[j]$ is less than pivot, we cannot move i and j anymore, we swap them

Low				High			
50	30	10	40	80	20	90	70
Pivot			i	j			

Continue moving i forward and j forward till the condition fails.

At i on 80, condition one fails

At $j=20$, condition two fails

No further movement, we swap 20 and 80

Low				High			
50	30	10	40	20	80	90	70
Pivot			i	j			

Incrementing i and decrementing j further, we get $j < i$.

Stop here, i is currently after j .

At this stage, we swap pivot with $v[j]$.

Low				High			
50	30	10	40	20	80	90	70
Pivot			j	i			

Low				Pivot		High	
20	30	10	40	50	80	90	70
				j	i		

Now, elements on the left of pivot are less than pivot. Elements on the right are more than pivots.

Recursively call quicksort on the left and right subarray. The subarrays are operated on like original arrays.

Algo

Algorithm QuickSort($A[0...n]$, low, high)

```
{
    if(low<high) then
        mid = partition( $A[low...high]$ )
        QuickSort( $A[low...mid-1]$ )
        QuickSort( $A[mid+1...high]$ )
}
```

```

}
Algorithm Partition(A[0...n], low, high)
{
    pivot=A[low];
    i = low;
    j = high+1;
    while(i ≤ j)do
    {
        while(A[i] ≤ pivot)do
        { i++;}
        while(A[j] ≥ pivot)do
        { j--;}
        if(i <= j) then
            swap(A[i],A[j])
    }
    swap(A[low],A[j])
    return j;
}

```

Time complexity

1. Best case: $T(n) = T(n/2) + T(n/2) + cn$
2. Using Master Theorem: check merge sort solution

$$T(n) = 2 * T(n/2) + cn$$

$$T(n) = \Theta(n \log n)$$

3. If pivot is a maximum and minimum of all the elements in the sorted list.

This implies if the pivot divides the array into a very skewed form: one part has 1 element and the other has n-1

$$T(n) = T(n-1) + n \quad \text{for } n > 1$$

$$T(n-1) = T(n-2) + (n-1)$$

$$T(n) = T(n-2) + (n-1) + n$$

And so on till $T(n) = T(1) + 2+3+4...$

Sum of first n natural numbers formula

$$T(n) = O(n^2)$$

4. Average Case

3. Average Case:

- For any pivot position i ; where $i \in \{0, 1, 2, 3 \dots n-1\}$
 - Time for partition an array: cn
 - Head and Tail sub-arrays contain i and $n-1-i$ items.
- So,

$$T(n) = T(i) + T(n-1-i) + cn$$

- Average running time for sorting:

$$T(n) = \frac{1}{n} \sum_{i=0}^{n-1} (T(i) + T(n-1-i)) + cn$$

Strassen's Matrix Multiplication

What happens:

1. Instead of multiplying regularly, we break A and B into four submatrices, multiply these together, and combine the result.
8 matrix multiplications on submatrices of A and B (four on each), plus $O(n^2)$ scalar operations of addition.

$$T(n) = 8T(n/2) + (n^2)$$

$$T(2) = O(1)$$

According to master theorem:

$$T(n) = O(n^3)$$

2. Found a way to do only 7 multiplications.

New runtime:

$$T(n) = 7T(n/2) + (n^2)$$

Applying master theorem with $a=7$, $b=2$, and $f(n)=(n^2)$,

$$T(n) = O(n^{2.81})$$

Algo:

```
Algorithm MatMul(A, B, n){
```

```
    if(n<=2) return
```

```
    C11 = a11xb11 + a12xb21
```

```
    C12 = a11xb12 + a12xb22
```

```
    C21 = a21xb11 + a22xb21
```

```
    C22 = a21xb12 + a22xb22
```

```
}
```

```
MatMul(A11, B11, n/2) + MatMul(A12, B21, n/2) //here, A11 and B11 are the first  
submatrix of both A and B
```

```
MatMul(A11, B12, n/2) + MatMul(A12, B22, n/2)
```

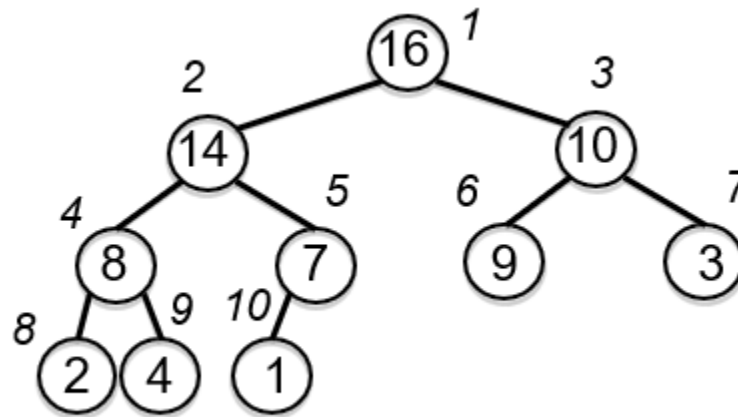
```
MatMul(A21, B11, n/2) + MatMul(A22, B21, n/2)
```

```
MatMul(A21, B12, n/2) + MatMul(A22, B22, n/2)
```


Heap sort

Concept of heap:

1. Priority queue: A data structure that maintains a Set S of elements. Each element is associated with a key.
2. $\text{insert}(S, x)$: inserts an element into the heap
 $\text{max}(s)$: returns max value in the heap
 $\text{extract_max}(S)$: returns max value and removes it from priority queue
 $\text{increase_key}(S, k, x)$: increase value at position x to k , only works if $k \geq s[x]$
 $\text{decrease_key}(S, k, x)$: decrease value at position x to k . Only works if k is less than $s[x]$
3. Max heap property: Each parent element's key is more than its children.



4. Heap as a tree
If root is i , the parent is at $i/2$
Left child is at $2i$
Right child is at $2i+1$
5. Heap operations
 - Max_heapify: correct a single violation of the heap property in a subtree at its root. If element $A[i]$ violates the max-heap property, correct violation by “trickling” element $A[i]$ down the tree, making the subtree rooted at index i a max-heap. Keep exchanging the violatee with the violator and call recursively till condition is met.
Complexity: $O(\log n)$
 - Build_max_heap: produce a max-heap from an unordered array.
Complexity: $O(n)$
 - Insert: add element to tree
 - Extract_max: Remove the highest key in tree
 - Heapsort: sort

Max_heapify pseudocode

```
Max_Heapify(A,i):  
  l = left(i)  
  r = right(i)  
  if (l <= heap-size(A) and A[l] > A[i])  
    then largest = A[l]  
  else largest = i  
  if (r <= heap-size(A) and A[r] > A[largest])  
    then largest = A[r]  
  if largest != i  
  then exchange A[i] and A[largest] Max_Heapify(A, largest)
```

Build max heap

```
Build_Max_Heap(A):  
  heap-size(A) = length(A)  
  for i=n/2 down to 1  
  do Max_Heapify(A, i)
```

Heapsort

1. Build Max Heap from unordered array
2. Find maximum element A[1]
3. Swap A(n) and A(1), now max el is at the end
4. Discard Node n
5. New root may violate max heap property, but its children are max heaps.
Run max_heapify to fix this.
6. If !heap.empty, repeat step 2 onwards.
7. Heap is empty after n iterations, each iteration takes log n time owing to max_heapify and swap operation. Hence total time complexity: $O(n \log n)$

Greedy

1. Used to solve optimization problems
2. Decision is taken based on currently available information
3. Greedy is based on determining the feasible solution which may or may not be the optimal solution.
4. Globally optimal solutions can be arrived at by making a locally optimal choice.
5. **Candidate function**: The set of all possible elements is known as a configuration set, or a candidate set from which the solution is created
6. **Selection function**: The selection function tells which of the candidates is the most promising. [which element to pick]
7. **Feasible Solution**: For a given input n , we need to obtain a subset that satisfies some constraints. Then any subset that satisfies these constraints is called a feasible solution. Checks whether the selected function is valid.
8. **Optimum solution**: The best solution from the feasible solution satisfying the objective function. [which is the best among all feasible solutions]
9. **Objective function**: This gives the rule to calculate the final value of the solution. It is a constraint that maximizes or minimizes the profit.

Algo

```
Greedy(int a[], int n)
{
    /* the array a[1]...a[n] contains the input */
    1. solution= $\phi$  /* initialise the solution */
    2. for(i=1;i<=n;i++)
    3. {
    4.     j=select(a);
    5.     if(feasible(solution,j))
    6. solution=union(solution,j);/* extend the
    solution */
    7. }
    8. return solution;
}/* End of Greedy(...) */
```

Applications

1. Knapsack

2. Job scheduling with deadlines
3. Single source shortest path dijkstra

Knapsack

What entails: Knapsack is a bag which can only hold w weight. There are n items of weights $w_1, w_2, \dots$ and profits $p_1, p_2, \dots$. The aim is to fill knapsack to maximise profit.

Knapsack problem is of two types: fractional knapsack and 0/1 knapsack

In a fractional knapsack, any fraction of an item can be added. If the total weight of a new item is say 10 and we only have room for 7 more items, we can add 7 parts of the item (a fraction of the total) and maximize profit. This is not possible in 0/1 knapsack, where we can only choose to take or not take the entirety of the item.

Fractional

What to do:

1. Compute profit value ratio of each item
2. Arrange in ascending order of profit value ratio
3. Fill from highest to lowest while knapsack capacity is achieved.

Algo:

Algorithm 1 FractionalKnapsack(S, W)

Input: Set $S = \{i_1, i_2, i_3, \dots, i_n\}$ of items. Each item is assigned with the value v_i and weight w_i . The maximum weight a knapsack can handle is W .

Output: Amount x_i of each item i to maximize benefit with weight at most W .

Repeat for each item $i = 1$ to n

set $x_i \leftarrow 0$ (selection part of item i is set to zero for every item initially)

set $p_i \leftarrow v_i / w_i$ {profit value of each item}

End repeat

set $cw \leftarrow 0$ and $i \leftarrow 1$ {set current total weight to 0 and start index to 1}

Repeat while $cw < W$ and $i \leq n$

remove item i with highest profit value p_i from set S

if $(cw + w_i) \leq W$ then

set $x_i \leftarrow 1$ and $cw \leftarrow cw + w_i$

else

set $x_i \leftarrow \{W - cw\} / w_i$

$cw = W$

endif

set $i \leftarrow i + 1$

endwhile

return(x)

Time complexity:

- Sorting: $O(n \log n)$
- Loop to maximise $O(N)$
- Total: $O(n + n \log n) = O(n (1 + \log n)) = O(n \log n)$

Space complexity: $O(1)$

0/1 Knapsack

In 0/1 knapsack, the items cannot be segregated into fractions. They're either selected or discarded. Greedy cannot be implemented in a 0/1 knapsack purely. This is because in greedy, once you make a local choice you cannot go back. In a 0/1 knapsack, there is no way to solidify a choice prematurely.

Three ways to implement 0/1:

1. Arrange in descending order of profit and fill till capacity is full
2. Arrange in ascending order of weight and pick in ascending order
3. Arrange in descending order of ratio and pick in that order (best)

-> more on this later under DP

Single source shortest path/Dijkstra

It is the algo to find the shortest path from a starting point in a weighted graph.

It creates a tree of shortest path from starting vertex to all other nodes in the path.

The Dijkstra's algo is a generalization of the BFS algorithm.

It does not work on negative weights.

Algo:

```

Algorithm Dijkstra ( $v_0$ ,  $W$ ,  $dist$ ,  $n$ )
//  $v_0$  is the source vertex,  $W$  is the adjacency matrix to store the weights of
the links,  $dist[k]$  is the array to store the shortest path to vertex  $k$ ,  $n$  is
the number of vertices//
{
    for ( $i = 1$  ;  $i \leq n$ ;  $i++$ ){
         $S[i] = 0$ ;                // Initialise set  $S$  to empty, i.e.,  $i$  is not
                                inserted into the set//
         $dist[i] = w(v_0, i)$ ;      //Initialise the distance to each node//
    }
     $S[v_0] = 1$ ;  $dist[v_0] = 0$ ;
    for ( $j = 2$ ;  $j \leq n$ ;  $j++$ ) {
        Choose a vertex  $u$  from those vertices which are not in  $S$  such that  $dist$ 
        [ $u$ ] is minimum.
         $S[u] = 1$ ;
        for (each  $z$  adjacent to  $u$  with  $S[z] = 0$ ) {
            if ( $dist[z] > dist[u] + w[u, z]$ )
                 $dist[z] = dist[u] + w[u, z]$ ;
        }
    }
}

```

Formula for dijkstra: If $\text{dist}[z] > \text{dist}[u] + w[u,z]$, then
 $\text{dist}[z] = \text{dist}[u] + w[u,z]$

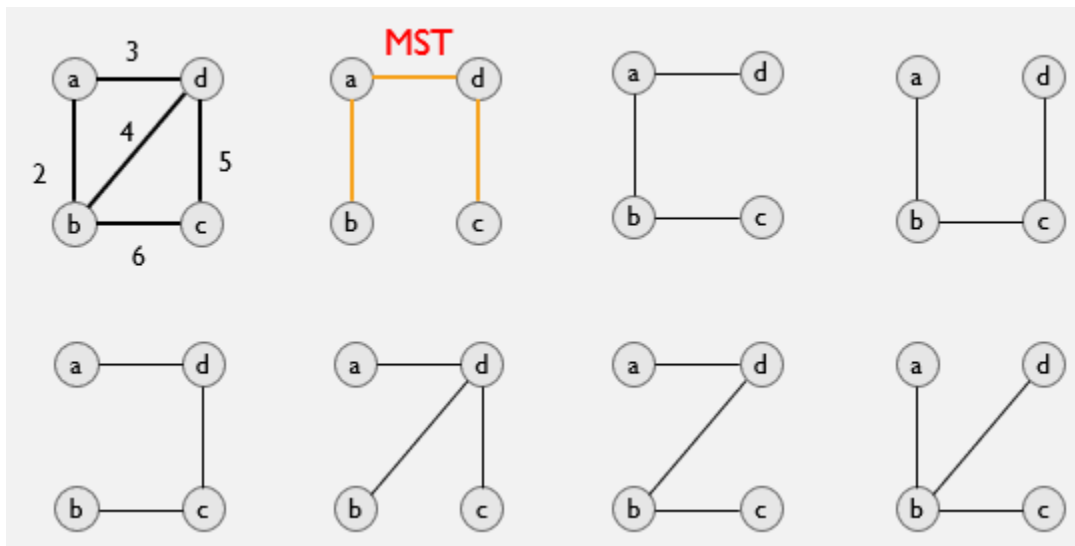
Time complexity: $O(n^2)$ where n is the number of vertices

Monkeybrain:

1. Make a priority queue with only src and dist 0 in it
2. Then pop this out in the first iteration and mark src visited. Using a priority queue ensures that everytime we pop the frontmost element, it is automatically the one with least distance.
3. Explore all its neighbors (v) and update their dist value as $\min(\text{dist}[v], \text{dist}[u] + w[u,z])$.
4. While exploring new unvisited neighbors, also add them to priority queue if they account for a change in $\text{dist}[v]$
5. Repeat till pq is empty

Minimum spanning tree

- A spanning tree is defined as a subgraph of an undirected graph in which all the vertices are connected.
- Weight of spanning tree = sum of all edge weights
- A spanning tree with the minimum total weight is called the MST: minimum spanning tree
- If there are N vertices in the graph, the MST must have $n-1$ edges
- Cycle = not possible



Here the first tree is chosen as mst because it has the least weight = $2+3+5 = 10$. It can be proven that another subgraph with lesser weight does not exist.

The rest are just spanning trees.

MSTs have two associated algorithms: Prims and Kruskal's

Prims algorithm

1. Select a node randomly
2. Iteratively add all connected nodes to this node into a tree
3. Identify a node closest to the tree and add minimum cost edge from that node to some node in the tree
4. Repeat with picking a node outside tree
With minimum edge to the tree.

Algorithm Prim(adjMatrix[1 .. n] [1 .. n])

***n** = Number of nodes in the graph*

***nEdges** = Current number of edges included in the MST.*

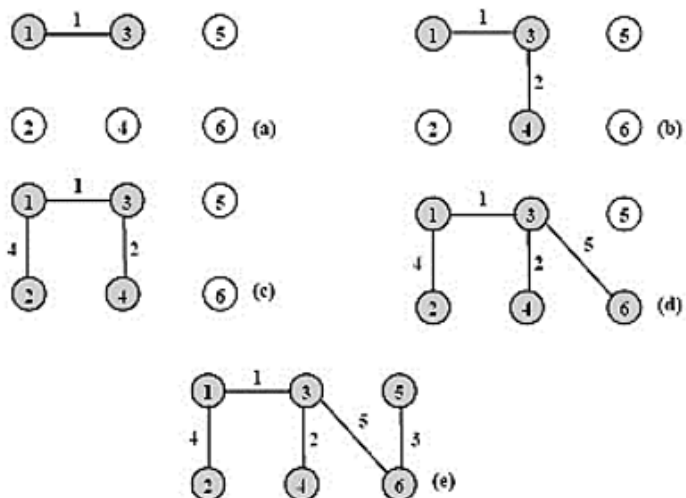
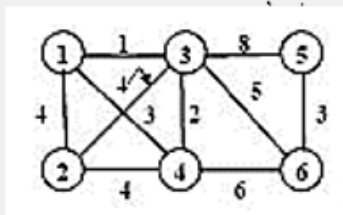
***min** = Minimum weight (cost) of edge.*

***row, col** indicate the row and column of the edge included in the MST.*

***sum** = Total minimum weight (cost) of the edges included in the MST.*

Initialize selected[] with zeros.

Example 1:



Kruskal's algorithm

1. Start with the edge with least cost
2. If multiple have least weight, select arbitrarily.
3. Add to spanning tree as long as no cycle is formed
4. Then add the edge with the next lowest cost and so on.

5. Repeat

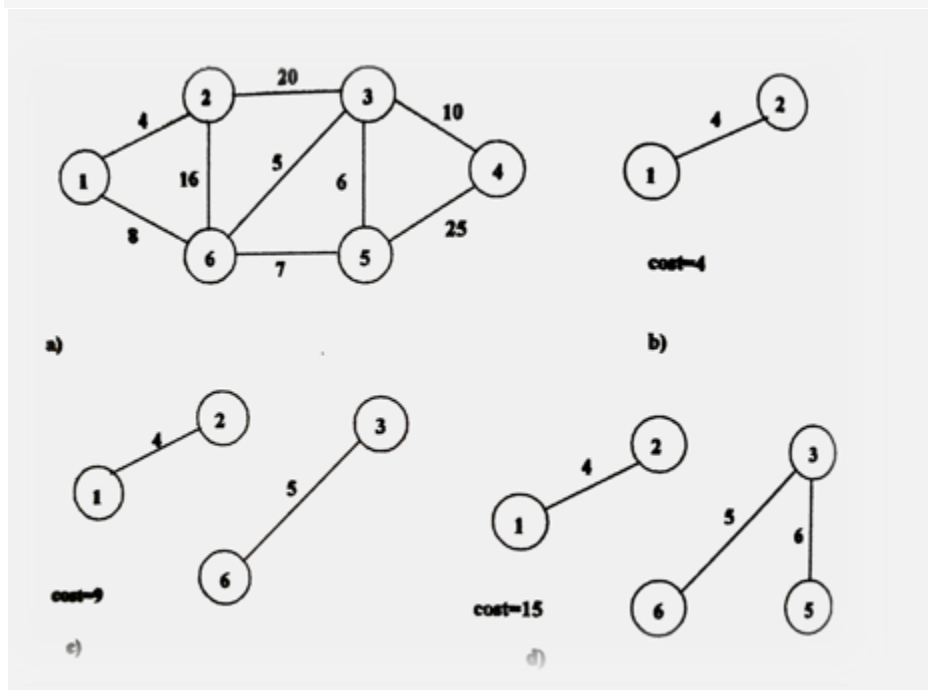
Algo

Algorithm Kruskal

tree = $\emptyset$;

While((tree has less than $n-1$ edges) && ($E \neq \emptyset$))

```
{
    select an edge (u,v) from E of lowest cost;
    remove(u,v) from E;
    if((u,v) does not create a cycle in tree)
        add(u,v) to tree;
    else
        discard(u,v);
}
```



Monkeybrain:

1. I go through all the edges one by one starting from the least weighted edge
2. If the nodes connected by this edge are already part of my tree, i ignore and move on
3. Else i add the weight to my mst and add the nodes to my tree
4. Using a disjoint set function (unimportant as to what this is but basically it checks if some nodes are connected in any way or form), i check if the current edge's source and destination are already in my tree. If they are, then they already confirm my connection and i do not need to add the weight.

5. If only one among the source or destination exists in the contending edge's tuple, we still add that edge to the tree and merge the components.
6. Continue till mst has $n-1$ edges

Coin change

A problem where amt A is to be achieved in minimum number of coins of denominations $A[]$.

Objective: Minimize the number of coins

Candidate: All possible solutions involving coins of all denominations

Selection: Largest denomination first, then move down when largest yet exceeds amt remaining.

Job sequencing with Deadlines

Problem:

1. There are n jobs
2. Each job has a deadline and a profit
3. Profit $[i]$ is attained if job is completed before deadline
4. Job is completed if it is processed on machine for 1 unit time
5. Only one machine is available at a time
6. Only one job may be processed at a time

Now what to do?

1. Sort jobs by profit cuz we greedy and we want most expensive job first
2. Let deadline for this job be t
3. Come back from t to 0 in our time array
4. Schedule this job for the latest time available in the subarray $[0, t]$
5. This way, we still have room for more jobs that might come later in the priority list before time while ensuring we get this job done before its deadline
6. If we dont find any empty slots for job j_1 in subarray $[0, t]$, we let this job go and check for incoming jobs.

Feasible: Subset of jobs J such that each job is complete before deadline

Optimal: Feasible soln with max profit

Consider the jobs in the descending order of profits

Algorithm 4.2: JobSequence($job[1..n], p[1..n], d[1..n], list[1..n], n$)		
1.	Initialize $list$ with zeros.	// make all n time slots empty
2.	$profit = 0$;	// $profit$ = total profit
3.	For $i = 1$ to n do	
4.	$k = d[i]$;	// set i th job's deadline to k
5.	While($k > 0$)	
6.	If($list[k] = 0$)	// if k th slot is vacant,
7.	$list[k] = job[i]$;	// assign i th job to k th slot
8.	add $p[i]$ to $profit$;	// accumulate total profit
9.	go to step 3;	// exit from while-loop (break)
10.	Endif	
11.	Decrement k by 1;	// otherwise get preceding slot
12.	Endwhile	
13.	Endfor	
14.	Print $list$	

Time complexity:

1. Sorting: $O(n \log n)$
2. For each job, we check time from t to 0 .
3. If $t = n$
4. Total time: $O(n \log n + n^2)$ where n^2 dominates so time = $O(n^2)$

Dynamic programming

Characteristics

1. Need an optimal substructure that makes decomposing the whole problem into subproblems easier.
2. Optimal solutions to the subproblem exist. These should branch out to solve the original problem.
3. When a recursive algorithm keeps revisiting the same subproblem, the problem has an overlapping subproblem.
4. **Principle of Optimality:** the optimal solution of a problem is a combination of optimal solutions to some of its subproblems.

For example, if we are given x queries about finding the factorial of x different numbers, we might have to run the same loop x times. But if we have previously calculated the factorial of a smaller number, we can just take its precomputed value the next time we need it instead of going all the way back to 1.

Applications

1. Longest common subsequence
2. 0/1 knapsack
3. Travelling salesman
4. Bellman ford etc

Homogeneous Recurrences

$$a_0 t_n + a_1 t_{n-1} + a_2 t_{n-2} + \dots + a_k t_{n-k} = 0$$

A recurrence of this form is linear, because it does not have a power higher than 1. It is homogeneous because the linear combination of t_{n-i} is zero. (RHS is zero and all a.t. terms add up to zero) It has constant coefficients like a .

Example:

$$t_n = t_{n-1} + t_{n-2}$$

Here, after adjusting

$$t_n - t_{n-1} - t_{n-2} = 0$$

$k=2$:

k = order of recurrence. It is the largest lag used. (Total number of terms-1)

$a_0=1, a_1=-1, a_2=-1$: coefficients of t_n, t_{n-1} etc

Property of Homogenous Recurrences

If $f(n)$ and $g(n)$ are solutions to an equation their linear combination is also a solution to the equation.

Proof:

$$a_0 f_n + a_1 f_{n-1} + a_2 f_{n-2} + \dots + a_k f_{n-k} = 0$$

$$a_0 g_n + a_1 g_{n-1} + a_2 g_{n-2} + \dots + a_k g_{n-k} = 0$$

$$t_n = c f_n + d g_n$$

$$t_n = a_0 (c f_n + d g_n) + a_1 (c f_{n-1} + d g_{n-1}) + a_2 (c f_{n-2} + d g_{n-2}) + \dots + a_k (c f_{n-k} + d g_{n-k})$$

$$= c(a_0 f_n + a_1 f_{n-1} + a_2 f_{n-2} + \dots + a_k f_{n-k}) + d(a_0 g_n + a_1 g_{n-1} + a_2 g_{n-2} + \dots + a_k g_{n-k})$$

$$= c(0) + d(0)$$

$$= 0$$

0/1 knapsack

1. $S[k, u] = 0$ if either $k=0$ or $u=0$
2. $S[k, u] = S[k-1, u]$, if u is less than w_k
3. $\max(S[k-1, u], S[k-1, u-w_k] + V_k)$ if u is not less than w_k

U = weight included in the knapsack thus far

w_k = weight of k 'th item

V = value of profit of K th item

Arrange information in increasing order of weight

Problem:

$W=5$

Item	1	2	3	4
Weight	2	3	4	5
Value	4	8	9	11

1. Fill row 1 and column 1 with 0 as per rule 1

2. At $S[1,1]$, U is 1 and it is less than $W_k = 2$
 $S[1,1] = S[0,1]$
3. $S[1,2]$, U is 2 and it is equal to $W_k=2$,
 Take max of prev and $S[k-1, u-w_k] + V_k$
 $S[1,2] = \max(0, 0+4) = 4$

weight-> item(k)\	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	4	4	4	4
2	0	0	4	8	8	12
3	0	0	4	8	9	12
4	0	0	4	8	9	12

Algorithm

Algorithm 5.1: knapsack($w[1..n]$, $v[1..n]$, W)

```

1.  For  $u = 0$  to  $W$ 
2.       $S[0, u] = 0$ 
3.  Endfor
4.  For  $i = 0$  to  $n$ 
5.       $S[i, 0] = 0$ 
6.  Endfor
7.  For  $i = 0$  to  $n$ 
8.      For  $u = 0$  to  $W$ 
9.          If(  $w[i] \leq u$  ) then    // item fits in knapsack
10.             If(  $v[i] + S[i-1, u-w[i]] > S[i-1, u]$  ) then
11.                  $S[i, u] = v[i] + S[i-1, u-w[i]]$ 
12.             Else  $S[i, u] = S[i-1, u]$ 
13.         Else  $S[i, u] = S[i-1, u]$ 
14.     Endfor
15. Endfor
16. Print  $S[n][W]$            // max value of items included in the knapsack

```

```

17 // Find knapsack items
18  $i = n, k = W$ 
19 While(  $i > 0$  and  $k > 0$  )
20     If(  $S[i][k] \neq S[i-1][k]$  )
21         Print  $i$  // item  $i$  is in the knapsack
22          $k = k - w[i]$ 
23     Endif
24      $i = i - 1$ 
25 Endwhile

```

Example:

Items: 1,2,3

Weight: 1,2,3

Profit: 6,10,3

weight->	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	6	6	6	6	6	6	6	6
2	0	6	10	16	16	16	16	16	16
3	0	6	10	16	16	16	19	19	19

Bellman Ford

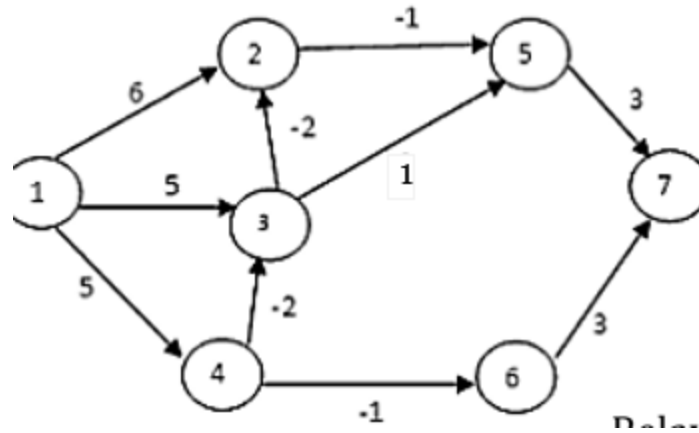
If graph has negative edge cost, then Dijkstra's Algorithm does not work. To solve this issue, we have the Bellman Ford Algorithm. It's slower than Dijkstra.

Relaxation formula:

```

if(  $d[v] > d[u] + \text{cost}(u,v)$  ) {
     $d[v] = d[u] + \text{cost}(u,v)$ 
}

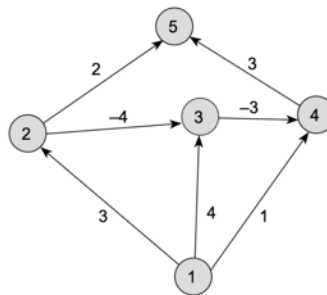
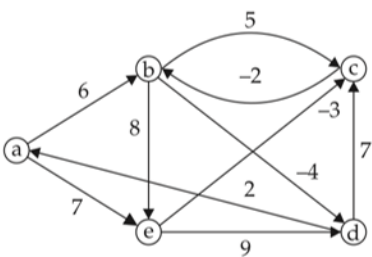
```



		1	2	3	4	5	6	7
Initialization	0	0	∞	∞	∞	∞	∞	∞
k=	1	0	6	5	5	∞	∞	∞
	2	0	3	3	5	5	4	∞
	3	0	1	3	5	2	4	5
	4	0	1	3	5	0	4	3
	5	0	1	3	5	0	4	3
	6	0	1	3	5	0	4	3

Here, $dp[i][j]$ represents the shortest path from 1 to j using maximum i edges

Practice



Algorithm

1. For each vertex v in V
 $D[v] = \infty$
2. $D[\text{source}] = 0$

3. For each edge (u, v) in E
 - If $D[v] > D[u] + W(u, v)$
 - $D[v] = D[u] + W(u, v)$

Algorithm 1: Bellman-Ford Algorithm

Data: Given a directed graph $G(V, E)$, the starting vertex S , and the weight W of each edge

Result: Shortest path from S to all other vertices in G

$D[S] = 0;$

$R = V - S;$

$C = \text{cardinality}(V);$

for each vertex $k \in R$ **do**

$D[k] = \infty;$

end

for each vertex $i = 1$ to $(C - 1)$ **do**

for each edge $(e1, e2) \in E$ **do**

$\text{Relax}(e1, e2);$

end

end

for each edge $(e1, e2) \in E$ **do**

if $D[e2] > D[e1] + W[e1, e2]$ **then**

$\text{Print}(\text{"Graph contains negative weight cycle"});$

end

end

Procedure Relax (e1, e2)

for each edge $(e1, e2)$ in E **do**

if $D[e2] > D[e1] + W[e1, e2]$ **then**

$D[e2] = D[e1] + W[e1, e2];$

end

end

Floyd Warshall Algorithm

Floyd's Algorithm is for finding the shortest path between every pair of vertices of a graph.

- Can be used in both directed and undirected graphs
- Negative edges can exist

- Negative cycles are forbidden. (total weight of cycle should be positive)
- Requires a weighted graph
- $O(n^3)$

Recursive formula:

$$D_k[i][j] = \min\{D^{k-1}[i][j], D^{k-1}[i][k] + D^{k-1}[k][j]\}$$

//practice dry run for one problem

Traveling Salesman Problem

Monkeybrain:

1. Salesman wants to visit all cities
2. No city should be visited twice
3. Salesman should return to source city
4. Cost is given (edge weight)
5. Goal is to find least cost

This problem satisfies optimal substructure property. It means the problem can be solved using solutions of smaller subproblems.

Recursive formula: $SP(i, S) = \min (d(i, k) + SP(k, S - \{k\}))$ for all k in S

Algorithm:

Algorithm 1: Dynamic Approach for TSP

Data: s : starting point; N : a subset of input cities; $dist()$: distance among the cities

Result: $Cost$: TSP result

$Visited[N] = 0;$

$Cost = 0;$

Procedure TSP(N, s)

```

    Visited[s] = 1;
    if |N| = 2 and  $k \neq s$  then
         $Cost(N, k) = dist(s, k);$ 
        Return Cost;
    else
        for  $j \in N$  do
            for  $i \in N$  and  $visited[i] = 0$  do
                if  $j \neq i$  and  $j \neq s$  then
                     $Cost(N, j) = \min ( TSP(N - \{i\}, j) + dist(j, i) )$ 
                     $Visited[j] = 1;$ 
                end
            end
        end
    end
    Return Cost;
end

```

Time Complexity:

For this dynamic programming TSP algorithm:

Number of subproblems:

Each state = (subset N, last city j)

Total subsets = 2^n

For each subset $\rightarrow$ n choices of j

So total states = $n \cdot 2^n$

Work per subproblem:

Inner loop tries all i in N

So work = $O(n)$

Total time complexity:

$T(n) = (\text{number of states}) \times (\text{work per state})$

$= (n \cdot 2^n) \times n$

$= O(n^2 \cdot 2^n)$

Space complexity:

Store DP table for all states

$= O(n \cdot 2^n)$

Time $\rightarrow O(n^2 \cdot 2^n)$

Space $\rightarrow O(n \cdot 2^n)$

Computation:

$$T = \sum_{k=1}^{n-2} (k-1)(n-1) C(n-2, k)$$

$$= (n-1) \sum_{k=1}^{n-2} (k-1) C(n-2, k)$$

Now,

$$\begin{aligned} & \sum_{k=1}^{n-2} (k-1) C(n-2, k) \\ &= \sum_{k=1}^{n-2} k C(n-2, k) - \sum_{k=1}^{n-2} C(n-2, k) \end{aligned}$$

Using identity:

$$k C(r, k) = r C(r-1, k-1)$$

$$= \sum_{k=1}^{n-2} (n-2) C(n-3, k-1) - \sum_{k=1}^{n-2} C(n-2, k)$$

$$= (n-2) \sum_{k=1}^{n-2} C(n-3, k-1) - (2^{n-2} - 1)$$

$$= (n-2)(2^{n-3}) - (2^{n-2} - 1)$$

Hence,

$$T = (n-1)[(n-2)(2^{n-3}) - (2^{n-2} - 1)]$$

$$= (n - 1)[(n - 2)(2^n)/8 - (2^{n/4} - 1)]$$

$$= (n - 1)(n - 2)(2^n)/8 - (n - 1)(2^{n/4} + (n - 1))$$

Final time complexity:

$$T = O(n^2 \cdot 2^n)$$

Matrix Chain Multiplication

Goal: we want to find most efficient way to multiply the given matrices together

Recursive function:

$$C[i, j] = \min_{i \leq k < j} \{C[i, k] + C[k + 1, j] + d_{i-1} * d_k * d_j\}$$

Algorithm:

```
Matrix-Chain(p, n)
{
  for (i = 1 to n) m[i, i] = 0;
  for (l = 2 to n)
  {
    for (i = 1 to n - l + 1)
    {
      j = i + l - 1;
      m[i, j] = ∞;
      for (k = i to j - 1)
      {
        q = m[i, k] + m[k + 1, j] + p[i - 1] * p[k] * p[j];
        if (q < m[i, j])
        {
          m[i, j] = q;
          s[i, j] = k;
        }
      }
    }
  }
  return m and s; (Optimum in m[1, n])
}
```

```
Mult(A, s, i, j)
{
  if (i < j)
  {
    X = Mult(A, s, i, s[i, j]);
    X is now Ai ··· Ak, where k is s[i, j]
    Y = Mult(A, s, s[i, j] + 1, j);
    Y is now Ak+1 ··· Aj
    return X * Y; multiply matrices X and Y
  }
  else return A[i];
}
```

Longest Common Subsequence

What is a subsequence? Subsequence of a sequence is the same sequence with some elements (possibly none) left out.

Algorithm:

```

lcs(char X[],char Y[],int c[][n],int m,int n)
{/* the two sequences are input in arrays X and Y. The length of the
longest common sequence is obtained in c[m][n] */
1.  for(i=0;i<=m;i++)
2.    c[i][0]=0;
3.  for(j=0;j<=n;j++)
4.    c[0][j]=0;
5.  for(i=1;i<=m;i++)
6.    for(j=1;j<=n;j++)
7.      if(X[i] != Y[j])
8.        c[i][j]=max (c[i-1][j],c[i][j-1]);
9.      else
10.        c[i][j]=c[i-1][j-1]+1;
11.  return c[m][n];
}/* End of lcs(...) */

```

Time complexity

Using Dynamic Programming:

Table size = $m \times n$

Each cell computed once

Time = $O(mn)$

Space complexity:

DP table = $O(mn)$

Multistage Graph

The multistage graph problem is to find the minimum cost path from a given source node to destination node.

There are two ways to solve:

Forward approach

Here we start from destination vertex(i.e. Sink)

$cost[i][j] = \min\{c[j][intermediate] + cost[i+1][intermediate]\}$

*for best understanding:

<https://github.com/ashwera/SYNotes/blob/master/Semester%202/Analaysis%20of%20Algorithms/aoa%20ost/multistageGraphFwd.cpp>

Algorithm 1 MultiGraphforward(G, k, n, p)

Input: The input is a k stage graph $G = \{V, E\}$ with $|V| = n$ vertices, E is a set of edges and $C[i][j]$ is the cost of edge (i, j) . $C[n][n]$ is the cost matrix of the given graph G . $Path[1..k]$ is the array of minimum cost path.

Output: The minimum cost path array P

```
1. Cost[n]=0;
2. For (j=n-1; j>=2; j--)
3. {
4. Cost[j]=C[j][r]+Cost[r]
   // Here r be a vertex such that <j, r> is an edge of G

5. d[j]=r
   // and(j, r) ∈ E and C[j][r] + Cost[r] is minimum
6. }
7. Path[1]=1
8. Path[k]=n
9. For (j= 2; j<=k-1; j++)
10. {
11. Path[j]=d[Path[j-1]]
12. }
13. Stop
```

Backward approach

Here we start from source vertex

$cost[i][j] = \min\{c[intermediate][j] + cost[i-1][intermediate]\}$

*best:

<https://github.com/ashwera/SYNotes/blob/master/Semester%202/Analaysis%20of%20Algorithms/aoa%20ost/multistageGraph.cpp>

Algorithm 2 MultiGraphbackward(G, k, n, p)

Input: The input is a k stage graph $G = \{V, E\}$ with $|V| = n$ vertices, E is a set of edges and $C[i][j]$ is the cost of edge (i, j) . $C[n][n]$ is the cost matrix of the given graph G . $\text{Path}[1 \dots k]$ is the array of minimum cost path.

Output: The minimum cost path array P

```
1. BCost[n]=0;
2. For (j=2; j<=n; j++)
3. {
4. BCost[j]= BCost[r]+ C[r][j]
   // Here r be a vertex such that <r, j> is an edge of G
5. Intermediate[j]=r
   // (j, r) ∈ E and BCost[r]+C[r][j] is minimum
6. }
7. Path[1]=1
8. Path[k]=n
9. For (j=k-1; j>=2 ; j--)
10. {
11.     Path[j]= Intermediate [Path[j+1]]
12. }
13.     Stop
```

Multistage graph complexity:

Let vertices = V

Let edges = E

We process stages from last to first.

For each vertex, check all outgoing edges.

Total work = sum of all edges processed once.

Time complexity = $O(E)$

Space complexity = $O(V)$

Bellman Ford vs. Multistage Graph

Multistage graph vs Bellman-Ford:

Multistage graph:

Graph is a Directed Acyclic Graph with stages.

Move only forward stage to stage.

No cycles allowed.

Each edge is processed once.

Time = $O(E)$

Bellman-Ford:

Works on any graph.

Cycles allowed.

Handles negative weights.

Relax all edges $V-1$ times.

Time = $O(VE)$

Multistage $\rightarrow$ staged, no cycles, fast $\rightarrow O(E)$

Bellman $\rightarrow$ general graph, repeats edges $\rightarrow O(VE)$

Backtracking and Branch and Bound

Both backtracking and branch and bound are based on search trees. In backtracking, we explore nodes in DFS manner and in branch and bound, we use the BFS method.

Backtracking

1. Modified DFS
2. Solves problems with two constraint types
 - a. Explicit: Rules that restrict every component of the solution vector
 - b. Implicit: rules that describe the way in which $x[i]$ should relate to each other.
3. State space tree: the solution space as a tree is called this.
4. Solution space: set of all tuple that satisfy the explicit constraint
5. Live node: node that has been generated by its descendants have not been generated yet.
6. Bounding function: Used to kill live nodes before its children are generated
7. Extended node: Live node whose children are being generated
8. Dead node: node that will not be extended further because it failed conditions or its children are all already generated
9. Answer node: node that represents the answer- implies that criteria have been met

10. Solution node: The node that has the potential to become the answer node.

N Queens

```
1  Algorithm NQueens( $k, n$ )
2  // Using backtracking, this procedure prints all
3  // possible placements of  $n$  queens on an  $n \times n$ 
4  // chessboard so that they are nonattacking.
5  {
6      for  $i := 1$  to  $n$  do
7      {
8          if Place( $k, i$ ) then
9          {
10              $x[k] := i$ ;
11             if ( $k = n$ ) then write ( $x[1 : n]$ );
12             else NQueens( $k + 1, n$ );
13         }
14     }
15 }
```

Algorithm 7.5 All solutions to the n -queens problem

```
1  Algorithm Place( $k, i$ )
2  // Returns true if a queen can be placed in  $k$ th row and
3  //  $i$ th column. Otherwise it returns false.  $x[ ]$  is a
4  // global array whose first ( $k - 1$ ) values have been set.
5  // Abs( $r$ ) returns the absolute value of  $r$ .
6  {
7      for  $j := 1$  to  $k - 1$  do
8          if ( $(x[j] = i)$  // Two in the same column
9              or ( $\text{Abs}(x[j] - i) = \text{Abs}(j - k)$ )
10             // or in the same diagonal
11             then return false;
12      return true;
13 }
```

Algorithm 7.4 Can a new queen be placed?

Better:

<https://github.com/ashwera/SYNotes/blob/master/Semester%202/Analaysis%20of%20Algorithms/aoa%20ost/nQueens.cpp>

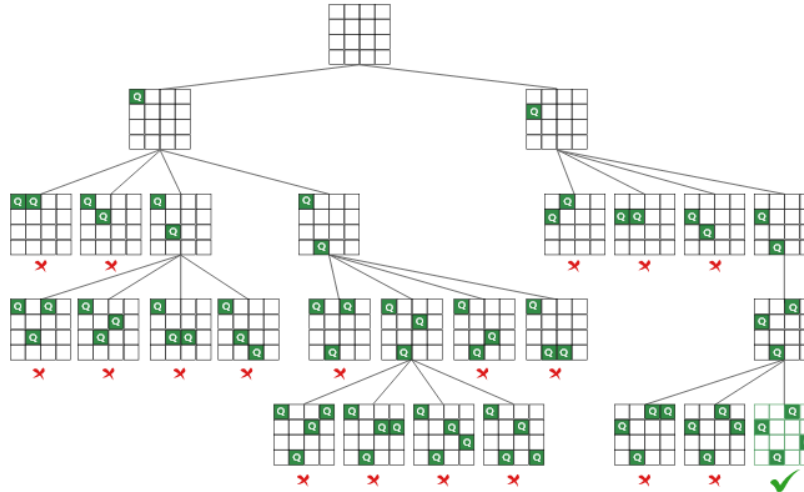
What happens:

1. For every cell in the each row, check if we can place a queen
2. If yes, mark it placed and update the solution tree.
3. Continue till either you reach the dead node or the answer node.

4. Then backtrack

5. How to check if a queen can be placed:

- Does there exist a queen in the same row/column?
- Is there a queen in the diagonal



Sum of Subsets

What the problem says:

You're given a target and an array.

Task: find number of subsets in that array that sum together to give target.

Approach:

- At each node of the binary tree, we either pick or leave the element
- It gives us two possible outlets
- At each level we decide to either take or leave a node.
- When the sum exceeds the target in any branch, we mark that node as dead and stop working on it.

<https://github.com/ashwera/SYNotes/blob/master/Semester%202/Analaysis%20of%20Algorithms/aoa%20ost/sumOfSubset.cpp>

Graph Coloring

What it entails:

- It's a problem of coloring N vertices of a given graph G in such a way that no two adjacent vertices share the same color and yet M colors are used.
- Modification: Smallest number of colors required to color the graph

Algorithm

- We represent Graph G by adjacency matrix $G[n,n]$ where, $G[i,j]=1$ if (i,j) is an edge between nodes i and j and $G[i,j]=0$ otherwise.
- If d is degree of given graph, then it can be colored with $d+1$ colors [m is referred to as chromatic number]. Degree = $2 \times \text{edges}$

```
Algorithm mColoring(k)
//The graph is represented by its
//boolean adjacency matrix G[1:n,1:n].
{
  repeat
  {
    NextValue(k);
    if(x[k]=0) then return;
    if(k=n) then
      write(x[1:n]);
    else mColoring(k+1);
  }until false;
}
```

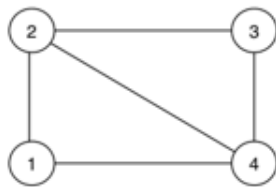
```
Algorithm NextValue(k)
{
  repeat
  {
    x[k] = (x[k]+1)mod(m+1);
    if(x[k]=0) then return;
    for j =1 to n do
    {
      if((G[k,j]≠0) and (x[k] = x[j])) then
        break;
    }
    if(j=n+1) then return;
  }until(false);
}
```

Better:

<https://github.com/ashwera/SYNotes/blob/master/Semester%202/Analaysis%20of%20Algorithms/aoa%20ost/graphColoring.cpp>

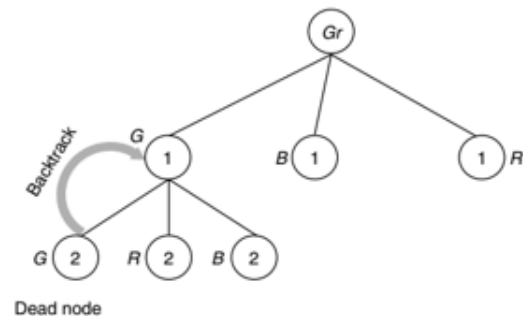
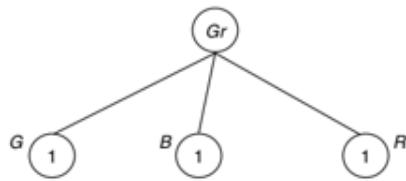
BACKTRACKING : Graph Coloring Example

Example 1:



	1	2	3	4
1	0	1	0	1
2	1	0	1	1
3	0	1	0	1
4	1	1	1	0

State Space Tree:



Explanation

1. Start with the first vertex.
Give it color 1.
2. Move to the next vertex.
Check its neighbors' colors.
Give it the smallest color not used.
3. Repeat for all vertices.
Always pick the lowest possible color.
4. If no color is available, use a new color.
5. End when all vertices are colored.
6. Example: if the neighbors are coloured 1 and 2, color the vertex 3 and so on.

Branch & Bound

Branch: using state space tree

Bound: using upper and lower bounds

The difference between this and backtracking is that in branch and bound, all children are generated before another live node becomes an E node.

The state space tree can be constructed using FIFO/LIFO and LC

FIFO

- Use a queue
- Node 1: e node

- The child nodes of the e node are generated. They are all placed in the queue
- The first element in the queue is repeatedly deleted, and its children are generated and pushed into the queue.
- This is continued till the answer node is found.

LIFO

- Stack is used
- Node 1 is taken as the e node initially
- The children of node 1 are generated and added to stack
- The first element in stack is deleted and its children are placed onto the stack
- This is continued till the answer is reached.

Branch and Bound FIFO/LIFO: Job scheduling with deadlines

Least Count (LC)

Chooses the best node

Implementation: Priority Queue

Problem: FIFO/LIFO are blind.

They don't choose smart nodes.

Idea: use cost (ranking function).

Steps:

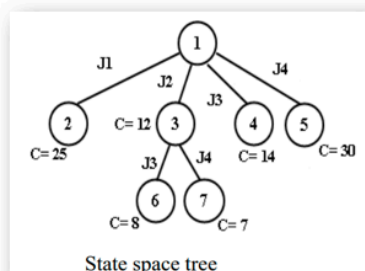
- Start with the root node. Calculate its cost.
- Generate child nodes. Compute cost for each child.
- Among all live nodes, pick nodes with minimum cost.
- Expand that node next.
- Repeat till solution found.

Example

LC (Least Count) Branch and Bound:

Example: Job sequencing with deadlines problem

- Jobs = {J1, J2, J3, J4}; P = {10, 5, 8, 3}; d = {1, 2, 1, 2}
- Initially we will take node 1 as E-node. Generate children of node 1, the children are 2, 3, 4, 5. By using ranking function we will calculate the cost of 2, 3, 4, 5 nodes is $\hat{c}=25$, $\hat{c}=12$, $\hat{c}=14$, $\hat{c}=30$ respectively.
- Now we will select a node which has minimum cost i.e., node 3. For node 3, the children are 6, 7.



Branch and bound is used with the 0/1 knapsack for the minimization problem. In this problem, the value of the objective function, $\sum p_i x_i$ is minimized subject to $\sum w_i x_i \leq M$. here $x_i = 0$ or 1 and $1 \leq i \leq n$

The two functions $c(x)$ or $U(x)$ are defined using two algorithms: Ubound and Bound.

Ubound: Computes the weights of objects in knapsack as a WHOLE. Their sum $\leq m$ and profit is reduced from initial profit and returned.

Bound: Computes the weights of objects in knapsack as FRACTIONS. Aims to use the entire capacity of the knapsack, $\sum w_i x_i = m$

Visualisation

0/1 knapsack

$$n = 4; m = 15;$$

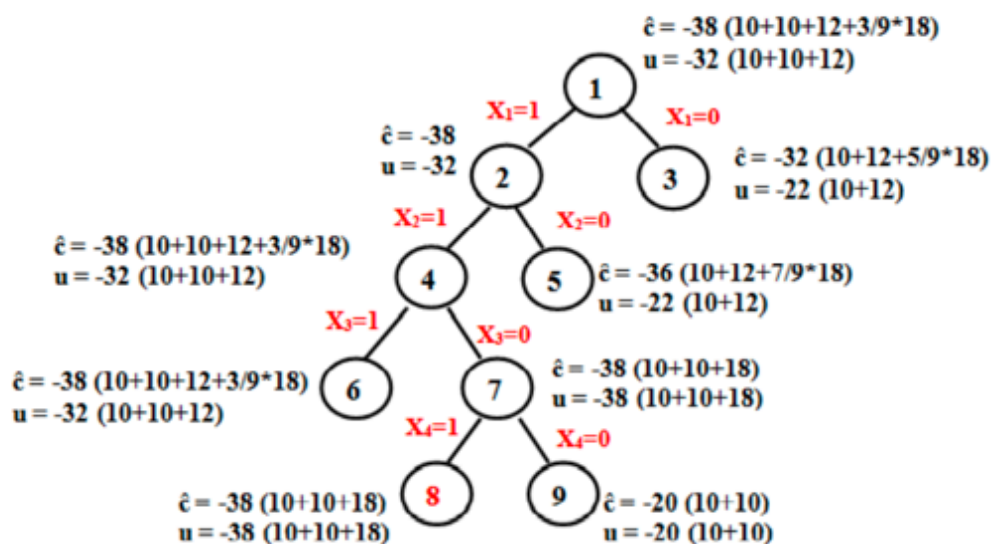
$$(p_1, p_2, p_3, p_4) = \{10, 10, 12, 18\}; (w_1, w_2, w_3, w_4) = \{2, 4, 6, 9\}$$

$$x_1 = 1,$$

$$x_2 = 1,$$

$$x_3 = 0,$$

$$x_4 = 1$$



In this graph, c represents the total profit following the fractional knapsack and u represents the total profit per the 0/1 knapsack.

We take the negative of the profit and with each $x_i=0$, we are disregarding that element from our knapsack. Aim is to maximise both c and u .

15 Puzzle Problem

Aim: there are 15 tiles placed arbitrarily on a 4*4 board numbered 1 to 15.

Task is to reach goal state by using all legal moves.

Goal state = numbers arranged in order

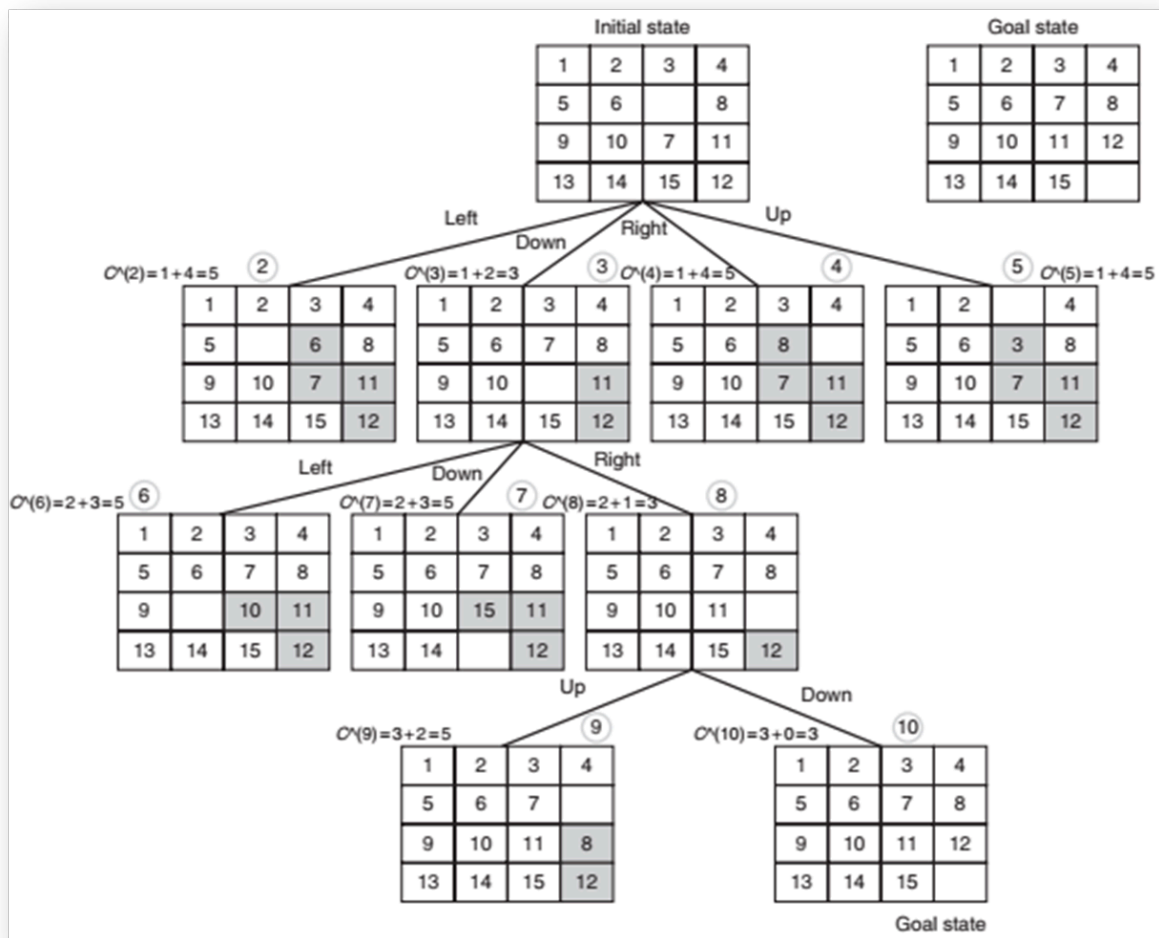
Legal move = a tile can only be swapped with the empty tile.

Initial state				Goal state			
1	2	3	4	1	2	3	4
5	6		8	5	6	7	8
9	10	7	11	9	10	11	12
13	14	15	12	13	14	15	

Figure 19 Initial and goal states for 15-puzzle problem.

- In the initial node, number of moves possible=4
- User can move any tile to the empty tile. From this step, we have four possibilities more
- Each move creates a new arrangement, hence a new state of the puzzle problem and a new node.
- By using all these states, a state space tree diagram is created.
- The LCBB method is the general method to solve 15 tiles.
- In state space tree, nodes are numbered as per the level. In each level we must calculate the value or cost of each node by using given formula: $C(x)=f(x)+g(x)$
Here, $f(x)$ = number of iteration in the space tree. Initial stage=0
 $g(x)$ = number of tiles not in proper place
- $c(x) < \text{infinity}$
- With each iteration, the smallest cost node is selected for further expansion. This node becomes the e-node.

Visualisation



Module 3

String Matching

String matching is a problem for searching a pattern within a text under conditions.

If a pattern P occurs after finite shift (cyclic movement) s in string T , then we can say s is a valid shift, otherwise invalid shift.

Example

If $T = \text{abcaabccbaab}$

And $P = \text{caabc}$

We know that the pattern caabc exists in T if we shift it from 2nd index

Naive String Matching

Naive string matching = compare substrings only
No actual shifting is happening

Algorithm/Pseudocode

```
int n = t.size();
int m = p.size();
for(int s=0; s<=n-m; s++){
    if(t.substr(s, m) == p){
        cout << "Pattern exists at shift " << s << endl;
    }
}
```

Complexity comparison

Algorithm	Preprocessing time	Matching time
Naive	0	$O((n - m + 1)m)$
Rabin-Karp	$\Theta(m)$	$O((n - m + 1)m)$
Finite automaton	$O(m \Sigma)$	$\Theta(n)$
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$

KMP

Better:

<https://github.com/ashwera/SYNotes/blob/master/Semester%202/Analaysis%20of%20Algorithms/aoa%20ost/kmp.cpp>

idea: avoid rechecking characters.

We preprocess the pattern using Longest Prefix Suffix.

LPS[i] = longest prefix = suffix till i.

Algorithm:

1. Start with two strings. Text T and Pattern P.
2. Build LPS array for P. LPS means longest prefix also suffix. It tells how much part of the text appears in both prefix and suffix.

Example:

Pattern = ababac

Index wise:

a $\rightarrow$ 0

ab $\rightarrow$ 0

aba $\rightarrow$ 1 (a)

abab $\rightarrow$ 2 (ab)

ababa $\rightarrow$ 3 (aba)

ababac $\rightarrow$ 0

LPS:

0 0 1 2 3 0

3. Take i for T, j for P in loop
4. If T[i] = P[j], move both indices forward
5. If mismatch:
 - o If j $\neq$ 0, set j = LPS[j-1]
6. Else move i

Pseudocode

```
KMP-MATCHER( $T, P$ )
1   $n = T.length$ 
2   $m = P.length$ 
3   $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P)$ 
4   $q = 0$  // number of characters matched
5  for  $i = 1$  to  $n$  // scan the text from left to right
6      while  $q > 0$  and  $P[q + 1] \neq T[i]$ 
7           $q = \pi[q]$  // next character does not match
8      if  $P[q + 1] == T[i]$ 
9           $q = q + 1$  // next character matches
10     if  $q == m$  // is all of  $P$  matched?
11         print "Pattern occurs with shift"  $i - m$ 
12          $q = \pi[q]$  // look for the next match
```

```
COMPUTE-PREFIX-FUNCTION( $P$ )
1   $m = P.length$ 
2  let  $\pi[1..m]$  be a new array
3   $\pi[1] = 0$ 
4   $k = 0$ 
5  for  $q = 2$  to  $m$ 
6      while  $k > 0$  and  $P[k + 1] \neq P[q]$ 
7           $k = \pi[k]$ 
8      if  $P[k + 1] == P[q]$ 
9           $k = k + 1$ 
10      $\pi[q] = k$ 
11  return  $\pi$ 
```

Longest Common Subsequence

Code:

<https://github.com/ashwera/SYNotes/blob/master/Semester%202/Analaysis%20of%20Algorithms/aoa%20ost/longestCommonSubsequence.cpp>

Algorithm:

1. LCS is a DP problem
2. We initialize a double dimensional dp array of size $n+1, m+1$ and populate it with zeroes.
3. Then we run a loop from $i=1$ to n and $j=1$ to m
4. When we encounter a match in the two strings, we update the dp array as $dp[i] = dp[i-1][j-1] + 1$;

5. Otherwise we take the max of $dp[i-1][j]$ and $dp[i][j-1]$ to fill in $dp[i][j]$
6. At the end, longest common subsequence length is stored in $dp[n][m]$.

What this does:

For each starting index in string s and t , it checks if there is a match. When a match is found, using pre-calculated values, it checks if a match was also found for an index anywhere before this index. If yes, then these two would add to the final subsequence, so we add its value to the answer. If no match is found YET, we add the longest found so far and continue working.

Time complexity

Time: $O(n * m)$

Reason:

Two nested loops run for all $n \times m$ states.

Backtracking takes $O(n + m)$.

Overall: $O(n*m)$

DP gives the longest length. But to actually construct the LCS, we go back from $dp[n][m]$ to $dp[0][0]$.

When a match is found, we go diagonal and when a match is not found, we go to the max between the cell above or cell to the left.

Module 4

Non-Deterministic Polynomial Algorithms

Almost all problems are of polynomial type. There are some that run in non-polynomial time also. But there is this separate set of problems called the NP problems. Once the correct solution is found, it's very fast to get to that solution. However, finding that correct solution itself is very difficult.

P and NP Classes

Polynomial

- Algorithm takes polynomial time to solve n^k
- If an algorithm is polynomial, we can formally define its time complexity as: $T(n) = O(C * n^k)$ where, $C > 0$ and $k > 0$ where C and k are constants and n is input size.
- Example: Matrix chain multiplication, insertion sort, merge sort, binary search, etc.
- If binary search has time complexity $\log n$, it's not of the form n^k , but it is still considered to be a polynomial type algorithm. This is because $\log n$ grows slower than any power of n .

Non Deterministic Polynomial

- The problems that are solved in polynomial time are called tractable problems. These include the ones discussed right above.
- The problems that require super polynomial time are called non-tractable problems. Super polynomial: $n^{\log n}$, 2^n , $n!$, they grow very fast.
- All deterministic polynomial time algorithms are tractable and the non-deterministic polynomials are intractable.
- When the outcome of a problem is not uniquely defined but can be limited to a specific set of possibilities, it's called a non-deterministic algorithm
- New statements to specify such algorithms.
 - choice(S): arbitrarily choose one of the elements of set S
 - failure: signals an unsuccessful completion
 - success: signals a successful completion
- The non-deterministic algorithm will terminate if there is NO set of choices that give us a successful completion.
- Example: Linear search
- Types of problems

- Any problem that answers in a yes or a no is called a decision problem and its answer is always called a decision algorithm
- Any problem that requires the identification of the maximum or minimum value of a given cost function is called the optimization problem and its answer is called the optimization algorithm.
- Example: traveling salesman has both a decision algorithm and an optimization algorithm
 - optimization: find Hamiltonian cycle of minimum weight
 - decision: is there a Hamiltonian cycle of weight $\leq k$.
- P is a subset of NP, where P is deterministic algorithms ka set and NP is the non-deterministic algorithms ka set.
- NP Hard: Every problem in NP can be converted into this one. So if you solve this, you indirectly solve all NP problems. But NP-hard problems are not required to be checkable fast. They can be even worse. Not every NP problem is an NP hard.
- A problem is NP complete if it is both NP and NP hard.

Example (only for understanding)

You have many puzzles where checking the answer is quick, like Sudoku. That is NP.

Now take 3-SAT. You are given a big logical formula. If someone gives values of variables, you can quickly check if it satisfies. But finding those values is hard. Also, every NP problem can be converted into this. So this is NP-complete. It is both hard and checkable.

Now take the Halting Problem. It asks whether a program will stop or run forever. This is even worse. You cannot even verify answers properly in general. It is harder than NP problems. So it is NP-hard but not NP-complete.

- Normally the decision problems are NP-Complete but the optimization problems are NP-Hard.

Reducibility of NP class

The NP complete problem set consists of many decision problems and nobody knows how to solve these in polynomial time. If we were to have a polynomial solution for any NP complete problem in this set, then every problem in this set would become solvable. To make this happen, we need the concept of reductions.

A problem A can be reduced to B if any instance of A can be rephrased as an instance of B.

When is a problem NP, NPH and NPC

1. When the output is given and you can verify the solution in polynomial time but cannot produce it: NP
2. A problem is NP-hard if all NP problems can be reduced to it in polynomial time.
3. We use reduction because we convert one problem into another in polynomial time: NP-complete = NP + NP-hard

Satisfiability problem

1. The satisfiability or the 3Sat problem is a boolean formula that can be constructed using literals and operations.
2. Literals are variables
3. Operations are boolean operations like and, or, not, xor, ec.
4. The satisfiability problem is used to determine whether a boolean formula is true or not. These are expressed in the CNF form.
5. For example if your cnf eq is $(a \wedge b \wedge c') \vee (a' \wedge b' \wedge c)$, we have a total of 8 possible values for the set.
6. Make state space tree for these three variables: each var can be either one or zero.

```
1  Algorithm Eval( $E, n$ )
2  // Determine whether the propositional formula  $E$  is
3  // satisfiable. The variables are  $x_1, x_2, \dots, x_n$ .
4  {
5      for  $i := 1$  to  $n$  do // Choose a truth value assignment.
6           $x_i := \text{Choice}(\text{false}, \text{true});$ 
7          if  $E(x_1, \dots, x_n)$  then Success();
8          else Failure();
9  }
```

Clique Problem

<https://www.youtube.com/watch?v=qZs767KQcvE>

1. A clique is a subgraph of a graph. The condition is that the subgraph should be complete. (all vertices are connected to all others)
2. The task of finding whether there is a clique of a given size in a graph (the clique decision problem) is NP-complete.

3. Maximal clique means a clique that you cannot extend by adding any more vertices. It is already “as big as possible” at that spot in the graph, even if a bigger clique exists somewhere else.
4. A maximum clique implies that there is no clique in the entire graph with more vertices than itself.
5. The clique decision problem is NP complete. **Statement:** Given an undirected graph G and a number k , does there exist a clique of size at least k , meaning a group of k vertices where every pair is connected.
6. Finding the maximum clique is intractable (very slow) and hard to approximate. Listing all maximal cliques requires exponential time.

For proving a clique in an NP, we take a graph and find its vertex cover.

<https://www.youtube.com/watch?v=qZs767KQcvE>

What is a vertex cover?

1. Pick some vertices and your vertex cover under the condition that every edge in the graph MUST be connected to at least one vertex cover.
2. Then, we remove these vertices from the graph.
3. The leftover vertices are called vertices in independent sets.
4. The smallest vertex cover gives the biggest independent set. This is because if your vertex cover is small, then a lot of vertices are leftover and they all become the independent set. And vice versa.
5. Total vertices - vertex cover size = independent set size.

Since the vertex cover is known to be NP-complete and the independent set is directly obtained from it, the maximum independent set is also NP-complete

Takeaway: if you take a satisfiability CNF formula with k clauses, then the graph made from it is bound to have clique of minimum size K

Vertex Cover Problem

Statement:

Takes a graph G and integer K as input.

Checks if there exists a vertex cover for G with at most K vertices or not.

Notice that vertex cover is in NP. Aim is to show that it is NP Hard.

Take 3-SAT $\rightarrow$ convert into graph $\rightarrow$ solve graph $\rightarrow$ get answer for 3-SAT $\rightarrow$ proves hardness.

We build a graph G and number k such that

If 3-SAT true, then graph has vertex cover $\leq k$

Step 1:

for each variable x_i , we have two vertices x_i and $\neg x_i$ joined by an edge.

We pick one vertex based on assignment:

if $x_i = \text{true} \rightarrow \text{pick } x_i$

if $x_i = \text{false} \rightarrow \text{pick } \neg x_i$

So all these edges are covered.

Step 2 (clauses):

Each clause is a triangle.

Since THE clause is true, at least one literal is true.

So we do not pick that one, and pick the other two vertices in triangle.

This still covers all edges of triangle.

Step 3 (connections):

Edges between clause and literals are also covered because the chosen literal vertices match assignment.

Result:

We selected exactly k vertices (n from variables + 2 per clause) and all edges are covered.

Monkeybrain for vertex cover:

1. Take any 3SAT problem (CNF)

Example: $(x_1 \vee \neg x_2 \vee x_3)$ AND $(\neg x_1 \vee x_2 \vee x_3)$

2. We have three variables, x_1 , x_2 , and x_3

3. For each variable, we have two vertices and we make an edge between them

a. $x_1 - \neg x_1$

b. $x_2 - \neg x_2$

c. $x_3 - \neg x_3$

Here $\neg x_3$ represents not x_3

4. We have two clauses:

$C_1 = (x_1 \vee \neg x_2 \vee x_3)$

$C_2 = (\neg x_1 \vee x_2 \vee x_3)$

5. For each clause, make a triangle (3 vertices, all connected).

6. We have total two triangles. Since we need a vertex cover (and therefore every edge must be connected to at least one vertex in cover), each triangle has at least two vertices in vertex cover.

7. Now we make a new graph altogether which depicts both point 3 and point 5. This implies that the new graph has two triangles and along with that its vertices are also connected to its complements. The triangle edges are called clause connections. The edges between a and its complement as shown in point 3 are called variable gadget. It forces you to pick one between the two.

8. C1 nodes connect to $x_1, \neg x_2, x_3$
9. C2 nodes connect to $\neg x_1, x_2, x_3$
10. Now we choose k . We decide how many vertices we are allowed to pick.
 $K = n + 2m$
 We check if the graph has a vertex cover of size $\leq k$
 Here n is the number of variables and m is the number of clauses
11. Pick N vertices that are true
12. For each clause, pick the other two vertices that make the triangle
13. This works because at least one vertex in every triangle is true. By picking the other two, the triangle is complete.

Reverse:

1. Meaning: suppose we already have a vertex cover of size k .
2. From variable pairs $(x_i - \neg x_i)$:
 we must pick at least one $\rightarrow$ that defines assignment
 pick $x_i \rightarrow x_i = \text{true}$
 pick $\neg x_i \rightarrow x_i = \text{false}$
3. From triangles:
 each triangle needs at least 2 vertices $\rightarrow$ so at least 1 is left out
 that leftover corresponds to a true literal
4. So every clause has at least one true literal $\rightarrow$ formula is satisfied
5. So: vertex cover $\rightarrow$ satisfying assignment exists

In a gist

You start with no vertices selected. Then you scan edges one by one. For each edge (u, v) , you check: is this edge already covered by something you picked earlier? If neither u nor v is in your set yet, then this edge is still uncovered, so you add both u and v to your vertex cover. That immediately covers this edge (and possibly others connected to u or v).

At the end, every edge has at least one endpoint in your set, so you have a valid vertex cover.

Function `Approx_vertex_cover(G)`

Step 1: set $V_c \leftarrow \emptyset$ (Initialize an empty vertex cover)

Step 2: for all $(u, v) \in E$

If $(u \in V_c \text{ and } v \in V_c)$

then set $V_c \leftarrow V_c \cup (u, v)$

Step 3: return V_c .

Questions

1. Implement merge sort on data
2. Explain greedy method terms.
Define candidate set, selection function, feasible solution, optimum solution, objective function.
Use one example to explain.
3. [graph given] implement and illustrate dijkstra and return the distance array.
4. What are homogeneous recurrences? List properties
5. Differentiate between Prim's algorithm and Kruskal's algorithm for MST with example. Which one is better? Justify your answer
6. Job scheduling algorithm
7. HIGH PRIORITY: 0/1 knapsack implementation with dp table
8. Bellman ford implementation making dp table when graph is given
9. Floyd warshall implementation when graph is given
10. Implement TSP and show derivation of time complexity
11. LCS implementation with table + algo
12. MCM algo
13. Fwd and bwd multistage. What is the difference between multistage and bellman ford
14. Differentiate between branch and bound and backtracking
15. unlikely: Define explicit constraint, implicit constraint, solution space, state space tree. Also define live node, dead node, extended node, bounding function, answer node.
16. Sum of subset / N queens: make search tree and write steps
17. Graph coloring: algo and steps
18. Visualise the 0/1 knapsack with branch and bound
19. 15 puzzle problem with branch and bound
20. Differentiate bw the naive and KMP algorithm for string matching.
Compare their complexity. Show demonstration.
21. Draw and explain LCS
22. What are asymptotic notations? Explain with example and graph.

All algorithms

All Algorithms

Dijkstra

```
Algorithm Dijkstra ( $v_0$ ,  $W$ ,  $dist$ ,  $n$ )
//  $v_0$  is the source vertex,  $W$  is the adjacency matrix to store the weights of
the links,  $dist[k]$  is the array to store the shortest path to vertex  $k$ ,  $n$  is
the number of vertices//
{
    for ( $i = 1$  ;  $i \leq n$ ;  $i++$ ){
         $S[i] = 0$ ;                // Initialise set  $S$  to empty, i.e.,  $i$  is not
                                // inserted into the set//
         $dist[i] = w(v_0, i)$ ;    //Initialise the distance to each node//
    }
     $S[v_0] = 1$ ;   $dist[v_0] = 0$ ;
    for ( $j = 2$ ;  $j \leq n$ ;  $j++$ ) {
        Choose a vertex  $u$  from those vertices which are not in  $S$  such that  $dist$ 
        [ $u$ ] is minimum.
         $S[u] = 1$ ;
        for (each  $z$  adjacent to  $u$  with  $S[z] = 0$ ) {
            if ( $dist [z] > dist [u] + w[u, z]$ )
                 $dist [z] = dist [u] + w[u, z]$ ;
        }
    }
}
```

Prims

for all vertices v
key[v] = ∞ , parent[v] = -1, seen[v] = false

key[start] = 0

Repeat n times:

pick ' u ' with minimum key
where seen[u] = false

mark seen[u] = true

for each adjacent vertex v of u
if seen[v] = false and $w(u,v) < key[v]$
then
key[v] = $w(u,v)$
parent[v] = u

Here, $\text{key}[x]$ is the minimum edge cost to connect v

Kruskal

Algorithm Kruskal

```
tree =  $\emptyset$ ;  
While((tree has less than  $n-1$  edges) && ( $E \neq \emptyset$ ))  
{  
    select an edge  $(u,v)$  from  $E$  of lowest cost;  
    remove $(u,v)$  from  $E$ ;  
    if(( $u,v$ ) does not create a cycle in tree)  
        add $(u,v)$  to tree;  
    else  
        discard $(u,v)$ ;  
}
```

Job scheduling with deadline

Algorithm 4.2: $\text{JobSequence}(\text{job}[1..n], p[1..n], d[1..n], \text{list}[1..n], n)$

1.	Initialize list with zeros.	// make all n time slots empty
2.	$\text{profit} = 0$;	// profit = total profit
3.	For $i = 1$ to n do	
4.	$k = d[i]$;	// set i th job's deadline to k
5.	While($k > 0$)	
6.	If($\text{list}[k] = 0$)	// if k th slot is vacant,
7.	$\text{list}[k] = \text{job}[i]$;	// assign i th job to k th slot
8.	add $p[i]$ to profit ;	// accumulate total profit
9.	go to step 3;	// exit from while-loop (break)
10.	Endif	
11.	Decrement k by 1;	// otherwise get preceding slot
12.	Endwhile	
13.	Endfor	
14.	Print list	

0/1 Knapsack

Algorithm 5.1: knapsack($w[1..n]$, $v[1..n]$, W)

```
1.  For  $u = 0$  to  $W$ 
2.       $S[0, u] = 0$ 
3.  Endfor
4.  For  $i = 0$  to  $n$ 
5.       $S[i, 0] = 0$ 
6.  Endfor
7.  For  $i = 0$  to  $n$ 
8.      For  $u = 0$  to  $W$ 
9.          If(  $w[i] \leq u$  ) then    // item fits in knapsack
10.             If(  $v[i] + S[i-1, u-w[i]] > S[i-1, u]$  ) then
11.                  $S[i, u] = v[i] + S[i-1, u-w[i]]$ 
12.             Else  $S[i, u] = S[i-1, u]$ 
13.             Else  $S[i, u] = S[i-1, u]$ 
14.         Endfor
15.     Endfor
16. Print  $S[n][W]$            // max value of items included in the knapsack
17 // Find knapsack items
18  $i = n, k = W$ 
19 While(  $i > 0$  and  $k > 0$  )
20     If(  $S[i][k] \neq S[i-1][k]$  )
21         Print  $i$            // item  $i$  is in the knapsack
22          $k = k - w[i]$ 
23     Endif
24      $i = i - 1$ 
25 Endwhile
```

Bellman Ford

Algorithm 1: Bellman-Ford Algorithm

Data: Given a directed graph $G(V, E)$, the starting vertex S , and the weight W of each edge

Result: Shortest path from S to all other vertices in G

$D[S] = 0;$

$R = V - S;$

$C = \text{cardinality}(V);$

for each vertex $k \in R$ **do**

$D[k] = \infty;$

end

for each vertex $i = 1$ to $(C - 1)$ **do**

for each edge $(e1, e2) \in E$ **do**

$\text{Relax}(e1, e2);$

end

end

for each edge $(e1, e2) \in E$ **do**

if $D[e2] > D[e1] + W[e1, e2]$ **then**

$\text{Print}(\text{"Graph contains negative weight cycle"});$

end

end

Procedure Relax (e1, e2)

for each edge $(e1, e2)$ in E **do**

if $D[e2] > D[e1] + W[e1, e2]$ **then**

$D[e2] = D[e1] + W[e1, e2];$

end

end

Traveling Salesman

Algorithm 1: Dynamic Approach for TSP

Data: s : starting point; N : a subset of input cities; $dist()$: distance among the cities

Result: $Cost$: TSP result

$Visited[N] = 0;$

$Cost = 0;$

Procedure TSP(N, s)

- $Visited[s] = 1;$
- if** $|N| = 2$ **and** $k \neq s$ **then**
 - $Cost(N, k) = dist(s, k);$
 - Return** $Cost;$
- else**
 - for** $j \in N$ **do**
 - for** $i \in N$ **and** $visited[i] = 0$ **do**
 - if** $j \neq i$ **and** $j \neq s$ **then**
 - $Cost(N, j) = \min (TSP(N - \{i\}, j) + dist(j, i))$
 - $Visited[j] = 1;$
 - end**
 - end**
- end**
- end**
- Return** $Cost;$

end

MCM

```
Mult( $A, s, i, j$ )
{
    if ( $i < j$ )
    {
         $X = \text{Mult}(A, s, i, s[i, j]);$ 
         $X$  is now  $A_i \cdots A_k$ , where  $k$  is  $s[i, j]$ 
         $Y = \text{Mult}(A, s, s[i, j] + 1, j);$ 
         $Y$  is now  $A_{k+1} \cdots A_j$ 
        return  $X * Y$ ; multiply matrices  $X$  and  $Y$ 
    }
    else return  $A[i]$ ;
}
```

```
Matrix-Chain( $p, n$ )
{
    for ( $i = 1$  to  $n$ )  $m[i, i] = 0$ ;
    for ( $l = 2$  to  $n$ )
    {
        for ( $i = 1$  to  $n - l + 1$ )
        {
             $j = i + l - 1$ ;
             $m[i, j] = \infty$ ;
            for ( $k = i$  to  $j - 1$ )
            {
                 $q = m[i, k] + m[k + 1, j] + p[i - 1] * p[k] * p[j]$ ;
                if ( $q < m[i, j]$ )
                {
                     $m[i, j] = q$ ;
                     $s[i, j] = k$ ;
                }
            }
        }
    }
    return  $m$  and  $s$ ; (Optimum in  $m[1, n]$ )
}
```

LCS

```
lcs(char X[],char Y[],int c[][n],int m,int n)
/* the two sequences are input in arrays X and Y. The length of the
longest common sequence is obtained in c[m][n] */
1.  for(i=0;i<=m;i++)
2.      c[i][0]=0;
3.  for(j=0;j<=n;j++)
4.      c[0][j]=0;
5.  for(i=1;i<=m;i++)
6.      for(j=1;j<=n;j++)
7.          if(X[i] != Y[j])
8.              c[i][j]=max (c[i-1][j],c[i][j-1]);
9.          else
10.             c[i][j]=c[i-1][j-1]+1;
11.  return c[m][n];
}/* End of lcs(...) */
```

Multistage fwd

Algorithm 1 MultiGraphforward(G, k, n, p)

Input: The input is a k stage graph $G = \{V, E\}$ with $|V| = n$ vertices, E is a set of edges and $C[i][j]$ is the cost of edge (i, j) . $C[n][n]$ is the cost matrix of the given graph G . $Path[1 \dots k]$ is the array of minimum cost path.

Output: The minimum cost path array P

```
1. Cost[n]=0;
2. For (j=n-1; j>=2; j--)
3. {
4. Cost[j]=C[j][r]+Cost[r]
   // Here r be a vertex such that <j, r> is an edge of G

5. d[j]=r
   // and(j, r) ∈ E and C[j][r] + Cost[r] is minimum
6. }
7. Path[1]=1
8. Path[k]=n
9. For (j= 2; j<=k-1; j++)
10. {
11. Path[j]=d[Path[j-1]]
12. }
13. Stop
```

Multistage bwd

Algorithm 2 MultiGraphbackward(G, k, n, p)

Input: The input is a k stage graph $G = \{V, E\}$ with $|V| = n$ vertices, E is a set of edges and $C[i][j]$ is the cost of edge (i, j) . $C[n][n]$ is the cost matrix of the given graph G . $Path[1 \dots k]$ is the array of minimum cost path.

Output: The minimum cost path array P

```
1. BCost[n]=0;
2. For (j=2; j<=n; j++)
3. {
4. BCost[j]= BCost[r]+ C[r][j]
   // Here r be a vertex such that <r, j> is an edge of G
5. Intermediate[j]=r
   // (j, r) ∈ E and BCost[r]+C[r][j] is minimum
6. }
7. Path[1]=1
8. Path[k]=n
9. For (j=k-1; j>=2 ; j--)
10. {
11. Path[j]= Intermediate [Path[j+1]]
12. }
13. Stop
```

N queens

```
1  Algorithm NQueens( $k, n$ )
2  // Using backtracking, this procedure prints all
3  // possible placements of  $n$  queens on an  $n \times n$ 
4  // chessboard so that they are nonattacking.
5  {
6      for  $i := 1$  to  $n$  do
7      {
8          if Place( $k, i$ ) then
9          {
10              $x[k] := i$ ;
11             if ( $k = n$ ) then write ( $x[1 : n]$ );
12             else NQueens( $k + 1, n$ );
13         }
14     }
15 }
```

Algorithm 7.5 All solutions to the n -queens problem

```
1  Algorithm Place( $k, i$ )
2  // Returns true if a queen can be placed in  $k$ th row and
3  //  $i$ th column. Otherwise it returns false.  $x[ ]$  is a
4  // global array whose first  $(k - 1)$  values have been set.
5  // Abs( $r$ ) returns the absolute value of  $r$ .
6  {
7      for  $j := 1$  to  $k - 1$  do
8          if ( $(x[j] = i)$  // Two in the same column
9              or ( $\text{Abs}(x[j] - i) = \text{Abs}(j - k)$ ))
10             // or in the same diagonal
11             then return false;
12     return true;
13 }
```

Algorithm 7.4 Can a new queen be placed?

Graph colouring

```
Algorithm mColoring(k)
//The graph is represented by its
//boolean adjacency matrix G[1:n,1:n].
{
  repeat
  {
    NextValue(k);
    if(x[k]=0) then return;
    if(k=n) then
      write(x[1:n]);
    else mColoring(k+1);
  }until false;
}
```

```
Algorithm NextValue(k)
{
  repeat
  {
    x[k] = (x[k]+1)mod(m+1);
    if(x[k]=0) then return;
    for j =1 to n do
    {
      if((G[k,j]≠0) and (x[k] = x[j])) then
        break;
    }
    if(j=n+1) then return;
  }until(false);
}
```

Naive String matching

```
int n = t.size();

int m = p.size();
for(int s=0; s<=n-m; s++){
    if(t.substr(s, m) == p){
        cout << "Pattern exists at shift " << s << endl;
    }
}
```


KMP

```
KMP-MATCHER( $T, P$ )
1   $n = T.length$ 
2   $m = P.length$ 
3   $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P)$ 
4   $q = 0$  // number of characters matched
5  for  $i = 1$  to  $n$  // scan the text from left to right
6      while  $q > 0$  and  $P[q + 1] \neq T[i]$ 
7           $q = \pi[q]$  // next character does not match
8      if  $P[q + 1] == T[i]$ 
9           $q = q + 1$  // next character matches
10     if  $q == m$  // is all of  $P$  matched?
11         print "Pattern occurs with shift"  $i - m$ 
12          $q = \pi[q]$  // look for the next match
```

```
COMPUTE-PREFIX-FUNCTION( $P$ )
1   $m = P.length$ 
2  let  $\pi[1..m]$  be a new array
3   $\pi[1] = 0$ 
4   $k = 0$ 
5  for  $q = 2$  to  $m$ 
6      while  $k > 0$  and  $P[k + 1] \neq P[q]$ 
7           $k = \pi[k]$ 
8      if  $P[k + 1] == P[q]$ 
9           $k = k + 1$ 
10      $\pi[q] = k$ 
11  return  $\pi$ 
```

Sum of Subset

```
void sos(int i, int sum, vector<int>& a, int target) {

    if(sum == target) {
        // print subset
        return;
    }

    if(i == a.size() || sum > target)
        return;

    // include
```

```

subset.push_back(a[i]);
sos(i + 1, sum + a[i], a, target);

subset.pop_back();

// exclude
sos(i + 1, sum, a, target);
}

```

Floyd warshall

```

Recurrence: dist[i][j]=min(dist[i][j], dist[i][k]+dist[k][j])
for(int k=0;k<n;k++){
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            dist[i][j] =
            min(dist[i][j],
                dist[i][k] + dist[k][j]);
        }
    }
}

```

topic	what?	method
dijkstra	source to all nodes shortest path	greedy
prims	mst by checking vertices	greedy
kruskal	mst by checking edges	greedy
job scheduling with deadline	sort then optimize	greedy
0/1 knapsack	take or leave items whole	dp/branch and bound
bellman ford	shortest paths from one source vertex to all other vertices.	dp
fractional knapsack	take or leave items half or whole	greedy
traveling salesman	visit all nodes in least cost, return to source	dp
matrix chain multiplication	Find multiplication order with minimum scalar multiplications.	dp
LCS	find longest common subsequence in two strings	dp
multistage graph	Find shortest/cheapest path from source to destination.	dp
n queens	fit n queens on a chessboard and make all safe	backtracking/space tree

graph coloring	color graph in min number of colours such that no adj nodes have same color	backtracking/space tree
naive string matching	find pattern in string	brute
KMP	find pattern in string	precomputation/string matching
Floyd Warshall	Find shortest distance between every pair of vertices.	dp
sum of subset	find subsets that sum to target	backtracking